

Lab 7 - OpenClaw Security

Apply security hardening for OpenClaw: run a security audit, restrict channel access, set resource limits, and review agent identity files.

Lab environment: Hostinger VPS (from Lab 1) or Docker Desktop (Windows 10/11, macOS 12+, Ubuntu 22.04+)

Prerequisite: Lab 3 completed - at least one channel running. Estimated time: 30 minutes

Docker Desktop users: Prefix every openclaw command with `docker exec -it openclaw`.

Config files are at `/home/node/.openclaw/` inside the container (not `/root/.openclaw/`).

Security Checklist

#	Step	Purpose
1	Run security audit	Find common misconfigurations
2	Restrict channel access with pairing	Only approved users can chat
3	Never hardcode API keys	Use env vars or volume-mounted secrets
4	Set Docker resource limits	Prevent runaway containers
5	Review SOUL.md and AGENTS.md	Control agent identity and permissions
6	Enable audit logs	Track all agent activity
7	Rotate API keys regularly	Limit damage from leaks
8	Use HTTPS for webhook channels (VPS)	Encrypt data in transit

Step 1 - Run the Security Audit

```
# VPS
openclaw security audit

# Docker Desktop
docker exec -it openclaw openclaw security audit
```

For a deeper check including live gateway probes:

```
# VPS
openclaw security audit --deep

# Docker Desktop
docker exec -it openclaw openclaw security audit --deep
```

Review the output and address any flagged items.

Step 2 - Restrict Channel Access

OpenClaw uses pairing to control who can message the agent. Only approved users can interact with the bot.

See all pending pairing requests: ``bash

VPS

openclaw pairing list

Docker Desktop

docker exec -it openclaw openclaw pairing list ``

Approve a specific user (e.g. from Lab 3): ``bash

VPS

openclaw pairing approve telegram YOUR_PAIRING_CODE

Docker Desktop

docker exec -it openclaw openclaw pairing approve telegram YOUR_PAIRING_CODE ``

Any user who has not been approved will receive access not configured when they message the bot.

Step 3 - Never Hardcode API Keys

Bad - hardcoded in command: ``bash openclaw configure --section model

Then pasting key directly

```
**Good - store as environment variable first:**
```

```
**VPS / macOS / Linux:**
```

```
export OPENAI_API_KEY="sk-..." export FIRECRAWL_API_KEY="fc-..." echo 'export OPENAI_API_KEY="sk-..."'
>> ~/.bashrc source ~/.bashrc ``
```

```
Windows (PowerShell): ``powershell [Environment]::SetEnvironmentVariable("OPENAI_API_KEY","sk-...", "User")
``
```

```
Docker Desktop - pass at container start: ``bash MSYS_NO_PATHCONV=1 docker run -d \ --name openclaw \ -p
18789:18789 \ -v openclaw-data:/home/node/.openclaw \ -e OPENAI_API_KEY="sk-..." \ openclaw/openclaw:latest
``
```

Step 4 - Set Docker Resource Limits (Docker Desktop Only)

Prevent the container from using all system memory and CPU:

```
docker update openclaw \
  --memory 1g \
  --cpus 1.5
```

Verify: ``bash docker stats openclaw --no-stream ``

Step 5 - Review SOUL.md and AGENTS.md

These files define your agent's identity and permissions.

VPS: ``bash cat ~/.openclaw/SOUL.md cat ~/.openclaw/AGENTS.md ``

Docker Desktop: ``bash #docker exec openclaw cat /home/node/.openclaw/workspace/SOUL.md #docker exec openclaw cat /home/node/.openclaw/workspace/AGENTS.md ``

MSYS_NO_PATHCONV=1 docker exec openclaw cat /home/node/.openclaw/workspace/SOUL.md
MSYS_NO_PATHCONV=1 docker exec openclaw cat /home/node/.openclaw/workspace/AGENTS.md ``

Remove any capabilities you did not intentionally grant.

Step 6 - View Audit Logs

```
# VPS
openclaw logs --limit 50

# Docker Desktop
docker exec -it openclaw openclaw logs --limit 50

# Follow live:
# VPS
openclaw logs --follow

# Docker Desktop
docker exec -it openclaw openclaw logs --follow
```

Step 7 - Rotate API Keys

1. Generate a new API key in the provider's dashboard
2. Reconfigure OpenClaw:

```
# VPS
openclaw configure --section model

# Docker Desktop
docker exec -it openclaw openclaw configure --section model
```

3. Revoke the old key in the provider's dashboard

Step 8 - Use HTTPS for Webhook Channels (VPS Only)

If you expose the gateway via a public domain, always use HTTPS:

```
apt install certbot python3-certbot-nginx -y
certbot --nginx -d YOUR_DOMAIN
```

Verification

Check	Expected
openclaw security audit	No critical issues flagged
openclaw pairing list	Only approved users listed
docker stats openclaw --no-stream	Memory within limits
openclaw logs --limit 20	Recent agent activity visible

Troubleshooting

Symptom	Fix
Security audit shows API key in config	Re-configure using env vars - do not paste keys directly
Unwanted user can message the bot	They were auto-approved - remove via openclaw pairing list to review
Docker stats command errors	Confirm container is running: docker ps
SOUL.md not found	File is created after first agent conversation

Reference

- Security: <https://docs.openclaw.ai/gateway/security>
- Pairing: <https://docs.openclaw.ai/pairing>